Clock Skew Scheduling: Formulations and Algorithms

Wai-Shing Luk

October 28, 2026

Abstract

Clock skew scheduling (CSS) assigns clock arrival times so that every setup- and hold-time constraint is met while a secondary objective is optimized. This survey rests on one idea: the timing constraints of a synchronous circuit form a system of difference constraints, feasible when the timing constraint graph has no negative cycle. Period minimization, slack maximization, and yield-driven scheduling reduce to parametric potential problems solved by Bellman–Ford, Lawler, Howard, Karp, and the ellipsoid method. The survey extends the core with delay padding, statistical methods, GEV delay models, and clock-tree co-optimization, closing with idealizations, modelling assumptions, and open problems.

1 Introduction

Synchronous digital design rests on a global clock that must reach every sequential element—every flip-flop (FF) and latch—at the right instant. Because the clock distribution network has finite, unequal delays, the active clock edge does not arrive at all registers simultaneously. The difference in arrival times is called *clock skew*. For a data path from register i to register j, with arrival times u_i and u_j, the (useful) skew is

$$y_{ij} = u_i - u_j.$$

Two facts make skew a first-class design variable. First, only *relative* arrival times are observable: adding a common constant to every u_i leaves every constraint unchanged, and because the clock is periodic an overall phase shift carries no physical meaning. Hence only the *differences* y_{ij} matter. Second, the setup- and hold-time constraints that guarantee correct data capture bound each skew from above and below, so each data path defines an interval of admissible skews. Clock skew scheduling (CSS) is the problem of choosing the arrival times—equivalently the skews—so that every timing constraint is met and some secondary objective is optimized.

Historically, designers targeted *zero skew* ($u_i = u_j$), which is easy to reason about: a single worst-case clock period can be verified by ordinary static timing analysis (STA). Zero skew, however, throws away a valuable degree of freedom. *Useful-skew* design [1], [2], [3] deliberately introduces nonzero skew to borrow slack from non-critical paths and lend it to critical ones. The result is either a higher clock frequency, a larger timing margin, or—under process variations—a higher parametric yield.

1.1 Why clock skew scheduling?

The value of a skew schedule is easily underestimated, because a schedule is a *plan* rather than a physical artifact and it is therefore tempting to dismiss it as an academic exercise. The practical considerations below argue otherwise.

The flow order must respect skew scheduling. A skew schedule is only as good as the delays it is computed from, and those delays are not final until clock-tree synthesis (CTS) and signal routing are both complete. The two steps are mutually dependent: CTS consumes the slack budget that the schedule allocates, while routing changes the data delays that the schedule assumed. A flow that treats the schedule as a one-shot pre-CTS step therefore computes it from estimates that later change. The practical consequence is that the schedule must be re-derived after routing—or, equivalently, that routing must be made aware of the skew budget—and how often this loop is closed determines how much of the available freedom survives into silicon [4], [5].

Placement is the main lever. A clock tree can only equalize the delays that the register locations allow; if the placement is poor, no algorithm inside CTS recovers the imbalance, because that imbalance is a property of where the registers sit. Placement must therefore anticipate the scheduler, and the front-end tools that shape it must be aware of the skew budget [5]. The consequence is that useful-skew design is less a single algorithm than a *tool-chain contract*: placement, routing, and CTS must each be aware of, and cooperate with, the skew schedule [4].

A timetable, not a single departure time. The clearest intuition is a railway timetable. Trains do not all depart at the same instant, nor are they required to arrive simultaneously at every station; departures and arrivals are deliberately staggered to maximize the throughput of the network. Zero-skew clocking is the opposite policy—every register is clocked at the same instant—and it forgoes precisely the freedom that a timetable exploits. Useful skew is the timetable of the clock, and the schedulers in this article are the algorithms that build it.

Why the formulation survives increasing complexity. The mathematical core of the problem is fixed at the outset and does not change as the model grows. The governing invariant is that only *relative* arrival times matter: shifting every clock by the same amount is physically meaningless, just as it is immaterial whether a schedule starts on Christmas or on New Year's Day. This invariant is exactly the hypothesis under which the constraints form a system of difference constraints, so the negative-cycle feasibility test and the single-parameter machinery of Sections 4–8 survive the statistical refinements almost unchanged. The scalar bisection of the single-parameter case generalizes to a separation-based convex iteration when several parameters are present (Section 4.4). Constraints that fix an *absolute* time, such as interface timing at the chip boundary or a clock-domain crossing, do break the translation invariance and lie outside the flat model; they are discussed in Section 11. This stability is the strongest reason to study the problem in the abstract form developed below.

1.2 The central idea

This survey is organized around a single unifying observation:

> The timing constraints of a synchronous circuit form a *system of difference constraints*,
>
> $$u_i - u_j \le w_e \quad \text{for every arc } e = (j, i),$$
>
> and such a system is feasible if and only if the associated *timing constraint graph* (TCG) contains no *negative cycle*.

Consequently, the natural formulation places clock skew scheduling within *network optimization*. Let A be the node–edge incidence matrix of the TCG, u the vector of arrival times, and $y = Au$ the vector of skews. Then the timing problem is a *feasible potential problem*:

$$\underline{w} \;\le\; y = Au \;\le\; \overline{w},$$

and its optimization variants—minimum clock period, maximum slack, maximum yield—are all *parametric potential problems* of the form

$$\max\{\, \beta \in \mathbb{R} \;:\; y \le d(\beta),\;\; Au = y \,\},$$

where $d(\beta)$ is a monotone decreasing function of the scalar parameter β. When $d(\beta) = m - s\beta$ is affine with nonnegative denominators s, this is the *minimum cost-to-time ratio cycle* problem; when s is a constant it further reduces to the *minimum mean cycle* problem. Both admit fast combinatorial algorithms—Howard's policy iteration, Karp's characterization, and Lawler's parametric search—and the negative cycle returned by these algorithms is not merely a certificate of infeasibility but also the *most critical* set of paths, which is exactly the information a designer needs.

1.3 Scope and organization

The remainder of the article is organized as follows. Section 2 fixes notation, derives the setup- and hold-time constraints, and constructs the timing constraint graph. Section 3 develops the network-potential formulation and the negative-cycle feasibility certificate. Section 4 treats clock-period minimization and slack maximization as parametric shortest-path problems, and extends them to multiple parameters via the ellipsoid method. Section 5 generalizes the parametric formulation to convex and quasi-convex network problems, covering min-cost flow, cycle cancellation, line search, linear-fractional and statistical costs, and the barrier method. Section 6 covers delay padding and its physical configurations. Section 7 addresses yield-driven scheduling under process variations, including the EVEN, PROP, C-PROP, and FP-PROP methods. Section 8 extends the model to non-Gaussian, heavy-tailed delay distributions via the generalized extreme value (GEV) distribution. Section 9 compares the underlying algorithms and their complexities. Section 10 links the schedule to clock-tree synthesis, and Section 11 lists open problems.

This article is a *survey*: it organizes and explains known formulations and algorithms rather than reporting new experiments, and the quantitative speedups quoted below are those reported in the cited literature. Where the abstract model diverges from physical design—clock-tree power and area, discrete cell sizes, spatial correlation, and hierarchical timing—the discrepancy is stated explicitly and revisited in Section 11.

Relation to prior work. The difference-constraint/negative-cycle view of timing is classical.

It underlies the retiming formulation of Leiserson and Saxe [6], [7], the cycle bound on clock period [8], [9], the LP formulation of skew scheduling [1], and the graph algorithms of Shenoy et al. [10]. The equivalence between retiming and skew scheduling is made explicit by Sapatnekar and Deokar [11] and by Liu, Papaefthymiou, and Friedman [12]. The parametric and ratio-cycle machinery is due to Lawler [13], Megiddo [14], Karp and Orlin [15], and Young, Tarjan, and Orlin [16]. The statistical line of work starts with Neves and Friedman [2] and Kourtev and Friedman [17], and continues through the non-Gaussian and extreme-value models of Wang et al. [18] and Zhu et al. [19]. What the present article adds is a single derivation that runs through all of these, together with an explicit account of what the model does not capture.

2 Preliminaries: Timing Constraints and the Timing Constraint Graph

2.1 Local data paths and clock skew

Consider the register pair (i, j) with at least one combinational path from the output of register i to the data input of register j (Figure 1). Let D_{ij} be the largest delay over all such paths and d_{ij} the smallest; these extrema are generally attained by *different* paths, and it is the pair of extrema that matters, since every individual path must satisfy both of the constraints derived below. The two registers are clocked by edges whose arrival times at i and j are u_i and u_j; the skew seen by the pair is $y_{ij} = u_i - u_j$.

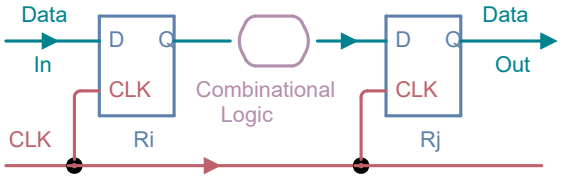


Figure 1: A local data path from register R_i to register R_j. The data traverses the combinational logic in a delay between d_{ij} and D_{ij}; only the skew $y_{ij} = u_i - u_j$ of the two clock arrival times matters.

Figure 2 shows the corresponding clock waveforms: the arrival times are simply the delays of the clock edges at the two registers, and the skew is their difference.

The circuit as a whole is abstracted as a directed graph in which vertices are registers and edges are data paths. A small example, together with its timing constraint graph, is shown in Figure 3.

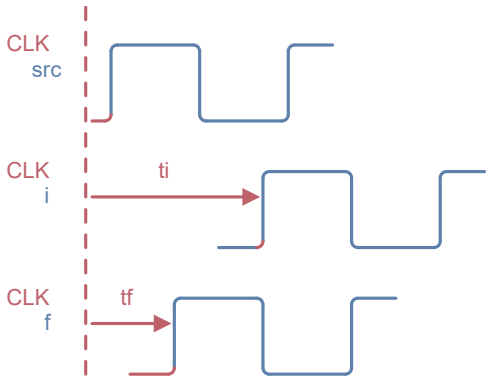


Figure 2: Clock waveforms. The source clock `CLK_src` and the clocks arriving at registers i and f are delayed by u_i and u_f; their difference $u_i - u_f$ is the skew.

2.2 Setup- and hold-time constraints

Correct capture requires two inequalities. The *setup-time* constraint says that data launched by i must reach j early enough before the capturing edge:

$$y_{ij} \;\le\; T_{\mathrm{CP}} - D_{ij} - T_{\mathrm{setup}}. \tag{1}$$

Here T_{CP} is the clock period and T_{setup} the setup time of register j. A violation of (1) is a *cycle-time violation*, also called *zero clocking*. The *hold-time* constraint says that new data must not arrive at j so early that it races past the previous capture:

$$y_{ij} \;\ge\; T_{\mathrm{hold}} - d_{ij}. \tag{2}$$

A violation of (2) is a *hold violation*, also called *double clocking* or a *short-path race*. Both T_{setup} and T_{hold} are effective guard bands rather than raw library values: in practice they also absorb clock jitter, PLL uncertainty, and on-chip-variation derating, so the bounds below should be read as the residual margins available to skew. Combining the two, the admissible skew of path $i \to j$ lies in the *feasible skew region* (FSR)

$$\begin{aligned} \underline{w}_{ij} \;\le\; y_{ij} \;\le\; \overline{w}_{ij}, \\ \underline{w}_{ij} = T_{\mathrm{hold}} - d_{ij}, \\ \overline{w}_{ij} = T_{\mathrm{CP}} - D_{ij} - T_{\mathrm{setup}}. \end{aligned} \tag{3}$$

Because $y_{ij} = u_i - u_j$, the constraints (3) are exactly a system of *difference constraints* on the arrival times u.

2.3 The timing constraint graph

The FSR (3) is represented by a directed graph $G(V, E)$, the *timing constraint graph*, one vertex per register:

- each *setup* constraint contributes an s-edge from j to i with weight $\overline{w}_{ij} = T_{\mathrm{CP}} - D_{ij} - T_{\mathrm{setup}}$;
- each *hold* constraint contributes an h-edge from i to j with weight $-\underline{w}_{ij} = d_{ij} - T_{\mathrm{hold}}$.

Indeed, the s-edge $j \to i$ encodes $u_i \le u_j + \overline{w}_{ij}$, i.e., (1); the h-edge $i \to j$ encodes $u_j \le u_i - \underline{w}_{ij}$, i.e., (2). A vertex assignment u satisfying every arc is precisely a *feasible potential*, and we have the fundamental equivalence:

> *The timing constraints are satisfiable by some clock schedule if and only if the timing constraint graph has no cycle of negative total weight.*

Summing the constraints around a cycle C eliminates the arrival times, because $\sum_{e \in C}(u_{\mathrm{head}} - u_{\mathrm{tail}}) = 0$; hence a cycle is feasible only if its weight is nonnegative. A negative cycle therefore certifies that *no* skew assignment can satisfy the corresponding constraints, and the only remedy is a circuit-level change—delay padding or logic restructuring (Section 6).

2.4 The zero-skew baseline and reported slacks

It is often convenient to start from a zero-skew analysis. If the clock is balanced so that $y_{ij} = 0$ for all paths, STA reports a *setup slack* S_{ij} and a *hold slack* H_{ij} for each path. Under useful skew, the constraints become

$$y_{ij} = u_i - u_j \le S_{ij}, \qquad -y_{ij} = u_j - u_i \le H_{ij}.$$

Since $S_{ij} = T_{\mathrm{CP}} - D_{ij} - T_{\mathrm{setup}}$ is exactly the upper skew bound, the reported slacks are the FSR bounds relative to the zero-skew solution. This is why an STA engine that reports slacks at zero skew is already sufficient as a front end: those slacks *are* the edge weights of the TCG. Two caveats apply in practice. First, a modern STA engine reports propagated-clock slacks, which already include the skew of the current tree; the zero-skew quantities must be recovered by subtracting the reported arrival-time differences. Second, STA reports the worst slack per endpoint rather than one slack per register pair, so reconstructing the TCG requires the path-level report rather than the endpoint summary.

3 The Network-Potential Formulation

3.1 Flow, potential, and duality

The TCG is a *network* in the sense of discrete calculus. Its node–edge incidence matrix A maps node potentials to edge tensions, $y = Au$, while its transpose A^{T} is the boundary operator that maps edge flows to nodal divergences. A *flow* x satisfies conservation $A^{\mathsf{T}}x = 0$ (a circulation); a *tension* y satisfies $\sum_{e \in C} y_e = 0$ around every cycle. Tellegen's theorem, $x^{\mathsf{T}}y = x^{\mathsf{T}}Au = (A^{\mathsf{T}}x)^{\mathsf{T}}u = 0$, states that flow and tension are bi-orthogonal—the two faces of the same network (Figure 4).

3.2 The feasible potential problem

In this language the timing problem is the *feasible potential problem*

$$\text{find } u \quad \text{such that} \quad \underline{w} \le y \le \overline{w}, \quad Au = y. \tag{4}$$

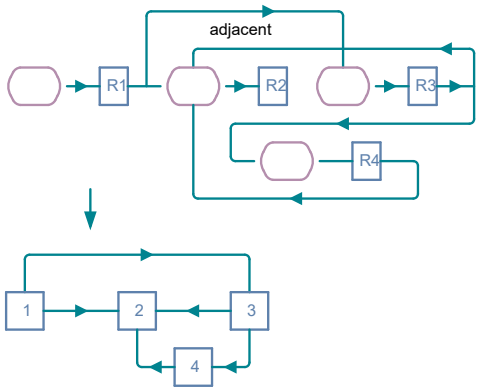
Feasibility of (4) has a clean combinatorial characterization. Because $y = Au$ is a tension, $\tau^{\mathsf{T}}y = 0$ for every circulation τ (a vector with $A^{\mathsf{T}}\tau = 0$). Writing τ^+ and τ^- for the positive and negative parts of τ, the box constraints give

$$\overline{w}^{\mathsf{T}}\tau^+ + \underline{w}^{\mathsf{T}}\tau^- \;\ge\; 0,$$

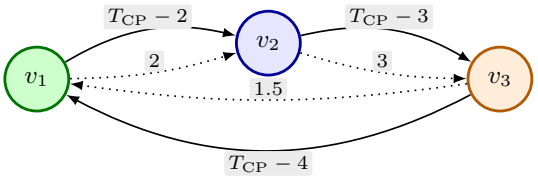
and this condition is also sufficient. Sufficiency is Hoffman's circulation theorem [20] applied to the graph obtained by splitting each edge into its upper and lower arcs; it is constructive, since the shortest-path computation that produces u is exactly the proof. For a simple directed cycle the condition reduces to the familiar span inequality. When only upper bounds are present—as in the timing constraint graph of Section 2, where the lower bounds have been folded into reverse arcs—it reduces to the requirement that every cycle have nonnegative weight, and an infeasible instance returns a *negative cycle*.

3.3 Bellman–Ford and the case for lazy evaluation

Difference-constraint systems have a long history in timing analysis and in temporal reasoning, where the



(a) Example circuit and its graph abstraction.



(b) Its timing constraint graph.

Figure 3: A circuit and its timing constraint graph. Solid arcs are setup constraints (s-edges); dotted arcs are hold constraints (h-edges). The weights are $T_{\mathrm{CP}} - 4$ and 1.5, etc.

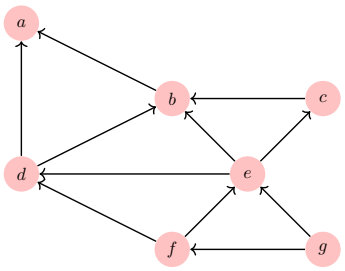


Figure 4: A flow (circulation) x and a tension (potential difference) y are dual objects on the same network.

same negative-cycle criterion appears under different names [21], [22], [23]. The classical instrument for this task is the Bellman–Ford algorithm [24], [25] (Algorithm 1). It computes shortest-path potentials and, in a final pass, either certifies feasibility or exhibits a negative cycle. Its simplicity is appealing, but it has practical drawbacks that matter in timing closure:

1. it is designed to find shortest paths, with negative-cycle detection only a by-product;
2. the basic pass structure reports infeasibility only after all $|V|$ passes have been completed;
3. it evaluates every edge weight on every pass.

The second and third points matter because, in a parametric or interactive loop, the weights are expensive to obtain: each one is an STA/SSTA query, and a tentative schedule may violate only a small subset of the constraints. Two standard remedies apply. A queue-based relaxation reports a negative cycle as soon as some vertex has been relaxed $|V|$ times, so the analysis stops at the first certificate instead of running to completion [26]. More importantly, when the weights depend on a parameter, only the arcs traversed while searching for a violated cycle need to be evaluated; the full set of weights is never materialized. This is the *lazy evaluation* principle used in Section 4.4. It is a statement about *evaluating weights*, not about knowing the graph, which must of course be given.

4 Clock Skew Scheduling: Feasibility and Period Minimization

4.1 Minimizing the clock period

A traditional objective is to find the smallest clock period T_{CP} for which a feasible schedule exists. This is the linear program (5),

$$\begin{aligned} &\text{minimize} && T_{\mathrm{CP}} \\ &\text{subject to} && \underline{w}_{ij} \le u_i - u_j \le T_{\mathrm{CP}} - D_{ij} - T_{\mathrm{setup}}, \\ & && \forall\, i \to j, \end{aligned} \tag{5}$$

whose constraints separate into a *hold part* independent of T_{CP} and a *setup part* that shrinks linearly as T_{CP} decreases. For a fixed T_{CP}, feasibility is decided by a single negative-cycle test on the TCG. Writing the cycle condition explicitly, a cycle C containing $k \ge 1$ setup edges is nonnegative exactly when

$$T_{\mathrm{CP}} \;\ge\; \frac{\sum_{e \in C^{\mathrm{s}}} (D_e + T_{\mathrm{setup}}) - \sum_{e \in C^{\mathrm{h}}} (d_e - T_{\mathrm{hold}})}{k}.$$

The smallest feasible period is therefore the *maximum*

Algorithm 1: Negative-cycle feasibility test (Bellman–Ford), with early exit when no relaxation occurs.

Input: TCG $G = (V, E)$ with weights w_e
Output: a feasible potential u, or a negative cycle

```
foreach v ∈ V do
    u_v ← 0;   π_v ← nil
for k ← 1 to |V| do
    changed ← false;
    foreach (p, q) ∈ E do
        if u_q > u_p + w_pq then
            u_q ← u_p + w_pq;   π_q ← p;   changed ← true;
    if not changed then return u;
foreach (p, q) ∈ E do
    if u_q > u_p + w_pq then
        return the cycle obtained by tracing π from q;
return u
```

over cycles of a ratio of a "cost" sum to a "time" sum. Negating the cost turns this into the *minimum* cost-to-time ratio cycle problem, which is the form solved by the cycle-based algorithms below; when the denominator is constant across edges it further reduces to the minimum mean cycle problem. This is the same cycle bound that retiming theory derives from a different argument [6], [8].

4.2 The parametric potential problem

All of the objectives above are instances of one *parametric potential problem*:

$$\max\{\, \beta \;:\; y \le d(\beta),\;\; Au = y \,\}, \qquad d(\beta) \text{ monotone decreasing.} \tag{6}$$

If $d(\beta) = m - s\beta$ is affine with $s \ge 0$, (6) is the minimum cost-to-time ratio cycle problem; if s is a constant, it is the minimum mean cycle problem. If $d(\beta)$ is nonlinear, (6) still makes sense but the linear combinatorial machinery must be replaced by binary search or a convex solver. If every f_e is non-increasing in β, the constraints only tighten as β grows, so the feasible set of β is a down-set: an interval $(-\infty, \beta^\star]$ (possibly empty). If, in addition, g is strictly increasing, then g attains its maximum at the right endpoint and $\beta^\star$ is the unique maximizer. Uniqueness refers to β alone: the potentials u are never unique, since the system is invariant under a common shift. If g is not monotone, or the feasible set is unbounded above, the problem may have no finite optimum.

4.3 Cycle-based algorithms

Two families of algorithms solve (6) directly.

Binary search (Lawler) [13]. Maintain an interval $[\beta_{\min}, \beta_{\max}]$ bracketing the optimum. For a trial β, test the TCG for a negative cycle; if one exists, the trial is too optimistic and the upper bound moves down, otherwise the lower bound moves up. Each test costs one Bellman–Ford pass, and the number of tests is logarithmic in the inverse tolerance. Lawler's method is simple and robust but can be slow.

Cycle cancellation (Howard) [27]. Howard's policy iteration maintains a schedule (equivalently, a set of shortest-path trees) and repeatedly finds a cycle along which the ratio can be improved, "zeroing out" that cycle with the smallest possible effort. It converges very fast in practice and returns the optimum together with *one* critical cycle. The gains of the intermediate policies are monotone, but the intermediate cycles are not themselves critical, and the algorithm has no polynomial worst-case bound; [28] reports it, together with Valdes' algorithm, as the fastest on circuit instances, while Young–Tarjan–Orlin is fastest overall. An improved variant introduces binary-search steps to accelerate convergence, and hybrid methods combine the strengths of both. The *sequence* of critical cycles, in decreasing order of criticality, is produced instead by the minimum-balancing algorithm of Section 4.

4.4 Multi-parameter problems and the ellipsoid method

Every formulation so far optimizes a *single* scalar parameter—the clock period, a uniform slack, or one

yield figure—and is therefore solvable by the combinatorial algorithms above. A real design, however, is often governed by *several* parameters at once: the clock period together with the timing slack, several yield targets, or the periods of multiple clock domains. The general problem is

$$\max\{\, g(\beta) \;:\; u_i - u_j \le f_{ij}(\beta) \;\; \forall (i,j) \in E \,\}, \qquad \beta \in \mathbb{R}^p, \tag{7}$$

where g and the f_{ij} are monotone in each component of β (and, for the convex extension, convex) but need not be affine. For $p = 1$ this is the parametric potential problem (6), solved by the minimum-ratio-cycle algorithms. For $p > 1$, parametric shortest-path theory no longer applies, and one must turn to a general convex optimization method.

The reason the problem stays tractable is that the procedure returning a negative cycle is a *separation oracle*. Given a trial β, either the timing constraint graph $G(\beta)$ is feasible, or it contains a negative cycle C. A negative cycle certifies infeasibility of the *fixed* β; to obtain a cut in β-space, one uses the convexity of the cycle sum. Let $h_C(\beta) = \sum_{e \in C} f_e(\beta)$; feasibility requires $h_C(\beta) \ge 0$ for every cycle. If h_C is convex and $h_C(\hat\beta) < 0$, then for any feasible β',

$$h_C(\beta') \;\ge\; h_C(\hat\beta) + g^{\mathsf T}(\beta' - \hat\beta), \qquad g = \nabla h_C(\hat\beta),$$

and since $h_C(\beta') \ge 0 > h_C(\hat\beta)$ the subgradient satisfies $g^{\mathsf T}(\beta' - \hat\beta) > 0$. The half-space $g^{\mathsf T}(\beta' - \hat\beta) \le 0$ therefore contains no feasible point, and g is a valid cut. An oracle thus needs only *one* violated cycle per iteration, and it evaluates only the arcs it traverses. This is the "lazy evaluation" principle of Section 3 lifted from feasibility to optimization.

The *ellipsoid method* [29], [30] is the natural engine for such an oracle: it maintains an ellipsoid known to contain the optimum, and each oracle call supplies a separating half-space through the current center, after which the ellipsoid is shrunk to the appropriate side until its volume (and hence the optimality gap) drops below a tolerance. Because it needs only separation, it tolerates non-differentiable objectives and nonlinear constraints; conceptually it is the multi-dimensional generalization of the bisection strategy used by Lawler's algorithm. For a central cut through β_k with normal g_k, the update is

$$\beta_{k+1} = \beta_k - \frac{1}{p+1}\,\frac{A_k g_k}{\sqrt{g_k^{\mathsf T} A_k g_k}}, \qquad A_{k+1} = \frac{p^2}{p^2-1}\Big(A_k - \frac{2}{p+1}\,\frac{A_k g_k g_k^{\mathsf T} A_k}{g_k^{\mathsf T} A_k g_k}\Big),$$

where A_k defines the current ellipsoid and p is the dimension of β. Each iteration reduces the volume by at least a factor $e^{-1/(2p)}$, so $O(p^2 \log(1/\varepsilon))$ iterations suffice for an ε-optimal solution; deep-cut and parallel-cut refinements reduce the constant in practice.

The same architecture is realized by the reference implementation of Section 11.2. On the linear problem the resulting solver has been reported to be faster than a general LP solver, and on the nonlinear problem substantially faster than a general convex solver [31]; those figures are inherited from that study and are not re-validated here.

4.5 Slack maximization

When no timing violation exists, the natural goal is to make the circuit as robust as possible by *maximizing the minimum slack*. The EVEN method (Section 7) is the simplest instance: maximize β subject to $u_i - u_j \le \mu_{ij} - \beta$, which is a minimum mean cycle problem. The optimal β is the amount of uniform safety margin that can be inserted into every constraint simultaneously. The *minimum balancing* (MB) algorithm [32] realizes this schedule constructively: it finds the most critical cycle, distributes the slack evenly along it, contracts it to a super-vertex, and repeats until a single vertex remains. The contraction order is itself valuable, because it records the *hierarchy of criticality* (Figure 5).

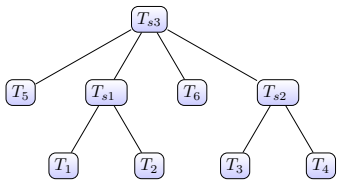


Figure 5: The criticality hierarchy produced by minimum balancing. Each internal node merges a critical cycle; deeper subtrees are less critical.

5 Convex and Quasi-Convex Network Problems

5.1 From a scalar parameter to a quasi-convex objective

This section is background rather than survey. It collects the network- optimization primitives that the clock-specific sections invoke—cycle cancellation, line search, and the barrier method—and states the structural correspondences that make them applicable, and flags where the analogy to timing stops.

Sections 4 and 4.4 reduced every scheduling variant to a parametric potential problem—(6) and (7)—in which the right-hand side $d(\beta)$ of a *scalar* parameter is monotone. Monotonicity, however, is a convenience rather than a requirement: the negative-cycle test that decides feasibility of $y \le d(z)$, $Au = y$ works for any right-hand side d. The convex-optimization viewpoint therefore enlarges the admissible objectives from monotone functions of a scalar to *convex* and *quasi-convex* functions of a vector z, and it supplies the separation oracle—a negative cycle—through which the enlarged problems are solved.

The bridge from a quasi-convex objective to a convex feasibility test is the *level-set* (sublevel-set) characterization. A function f is quasi-convex exactly when every sublevel set $\{z : f(z) \le \gamma\}$ is convex; equivalently, when there is a family of functions Φ_γ such that

- $\Phi_\gamma(z)$ is convex in z for every fixed γ,
- $\Phi_\gamma(z)$ is non-increasing in γ for every fixed z, and
- $f(z) \le \gamma$ if and only if $\Phi_\gamma(z) \le 0$.

For a ratio of a convex function to a positive concave function, $f(z) = p(z)/q(z)$ with p convex, q concave, $p \ge 0$, and $q > 0$, the choice

$$\Phi_\gamma(z) = p(z) - \gamma\, q(z)$$

has all three properties: it is convex in z for each fixed γ, and its 0-sublevel set is exactly the γ-sublevel set of f. Linear-fractional functions, distance ratios, and the minimum cost-to-time ratio cycle of Section 4 are all of this form.

Minimizing a quasi-convex f therefore reduces to a sequence of convex feasibility problems (8) parametrized by γ,

$$\begin{array}{ll} \text{find} & z, \\ \text{subject to} & \Phi_\gamma(z) \le 0, \\ & y \le d(z), \quad Au = y, \end{array} \tag{8}$$

in which the subproblem is feasible if and only if $\gamma \ge f^\star$ and infeasible otherwise; a bisection on γ converges to the optimum. Each subproblem is convex and its separation oracle is the negative-cycle test applied to the graph weighted by d: one violated constraint per iteration suffices, which is exactly the lazy-evaluation principle of Section 4.4 lifted from a scalar parameter to a vector.

A useful special case is *monotone min-max*,

$$\min_y \Big\{ \max_{ij} f_{ij}(y_{ij}) : Au = y \Big\},$$

with each f_{ij} non-decreasing. Introducing a scalar β and inverting the f_{ij} reconstitutes a monotone parametric potential problem,

$$\max\big\{\beta : y_{ij} \le f_{ij}^{-1}(\beta) \;\; \forall (i,j), \quad Au = y\big\},$$

because each inverse is non-decreasing in β. The max-min yield formulation of Section 7 is precisely of this type, with f_{ij} the quantile function of a path-delay distribution; the framework below is what makes its probabilistic and nonlinear variants solvable by the same negative-cycle machinery.

5.2 Min-cost flow and cycle cancellation

The same primitive drives *min-cost flow*. The linear problem is

$$\begin{array}{ll} \text{minimize} & d^{\mathsf T} x + p \\ \text{subject to} & c^- \le x \le c^+, \\ & A^{\mathsf T} x = b, \quad b(V) = 0, \end{array} \tag{9}$$

where x is an arc-flow vector, $A^{\mathsf T}$ the incidence matrix of G, c^- and c^+ (possibly infinite) capacity bounds, and b a balanced supply vector. Its classical algorithms fall into two families. *Augmenting-path* methods start from an infeasible flow and inject the smallest possible amount along an augmenting path, preserving infeasibility until no flow remains to inject. *Cycle-cancelling* methods [13], [33] start from a feasible flow x_0 and repeatedly set $x_1 = x_0 + \alpha\, \Delta x$, where Δx is the indicator of a negative cycle of the residual graph and $\alpha > 0$.

The second family is a descent method in disguise. Because Δx is a circulation, $A^{\mathsf T} \Delta x = 0$, a negative cycle with respect to the cost d satisfies $d^{\mathsf T} \Delta x < 0$, so moving along it decreases the cost. The step is limited by the residual capacities,

$$\alpha_1 = \min_{ij} \{ c^+_{ij} - x^0_{ij} : \Delta x_{ij} > 0 \}, \qquad \alpha_2 = \min_{ij} \{ x^0_{ij} - c^-_{ij} : \Delta x_{ij} < 0 \},$$

with $\alpha_{\text{lin}} = \min\{\alpha_1, \alpha_2\}$; if $\alpha_{\text{lin}} = +\infty$ the problem is unbounded. Choosing a *minimum mean* negative cycle rather than an arbitrary one accelerates convergence. Cycle cancellation is thus the network specialization of a generic descent method: choose a descent direction, choose a step size, update, and repeat.

This is the structure the clock-skew solvers exploit. The feasible-potential problem $\underline{w} \le Au \le \overline{w}$ is dual to a min-cost flow of the form (9), and the negative cycle returned by Howard's algorithm (Section 4) is simultaneously the infeasibility certificate and the descent direction. Delay padding (Section 6) is the primal min-cost-flow instance, whose dual variable x measures how the padding budget is spent.

5.3 Convex costs and line search

When the cost is a convex nonlinear function $f(x)$,

$$\begin{array}{ll} \text{minimize} & f(x) \\ \text{subject to} & 0 \le x \le c, \\ & A^{\mathsf{T}}x = b, \quad b(V) = 0, \end{array} \tag{10}$$

the same descent applies with the arc cost replaced by the gradient: choose Δx as a negative cycle of the residual graph for the cost $\nabla f(x)$, so that $\nabla f(x)^{\mathsf{T}}\Delta x < 0$. The step is now limited by capacities *and* by the line search,

$$\alpha_{\text{cvx}} = \min\{\alpha_{\text{lin}}, t\},$$

where t is either an exact step, $t = \arg\min_{t>0} f(x + t\,\Delta x)$, or a backtracking step with parameters $\alpha \in (0, 1/2)$ and $\beta \in (0, 1)$ that shrinks t while

$$f(x + t\,\Delta x) > f(x) + \alpha\, t\, \nabla f(x)^{\mathsf{T}}\Delta x.$$

The linear problem (9) is the special case in which $t = +\infty$ and only the capacity bound is active. This convex cycle-cancelling descent is the method implemented by general-purpose network-optimization libraries such as LEMON.

5.4 Linear-fractional and statistical costs

Two quasi-convex costs arise repeatedly in timing. The *linear-fractional* min-cost flow,

$$\min\Big\{\frac{e^{\mathsf{T}}x + f}{g^{\mathsf{T}}x + h} : 0 \le x \le c,\ A^{\mathsf{T}}x = b,\ b(V) = 0\Big\},$$

is recast, with $p(x) = e^{\mathsf{T}}x + f$ and $q(x) = g^{\mathsf{T}}x + h$, as the family

$$\min\big\{\gamma : (e - \gamma g)^{\mathsf{T}}x + (f - \gamma h) \le 0,\ 0 \le x \le c,\ A^{\mathsf{T}}x = b\big\},$$

solved by bisection on γ with descent directions of cost $e - \gamma g$. This is the minimum cost-to-time ratio problem of Section 4 expressed as a flow: the affine parametric weight $d(\beta) = m - s\beta$ there is the affine cost $e - \gamma g$ here.

The *statistical* problem

$$\min\Big\{\Pr\big(\mathbf{d}^{\mathsf{T}}x > \alpha\big) : 0 \le x \le c,\ A^{\mathsf{T}}x = b,\ b(V) = 0\Big\}$$

treats $\mathbf{d}$ as a random vector with mean d and covariance Σ, so that $\mathbf{d}^{\mathsf{T}}x$ has mean $d^{\mathsf{T}}x$ and variance $x^{\mathsf{T}}\Sigma x$. The chance constraint is quasi-convex and is recast as

$$\min\big\{\gamma : d^{\mathsf{T}}x + F^{-1}(1-\gamma)\,\|\Sigma^{1/2}x\|_2 \le \alpha,\ 0 \le x \le c,\ A^{\mathsf{T}}x = b\big\},$$

a second-order-cone (convex quadratic) constraint in x whose gradient is

$$d + F^{-1}(1-\gamma)\,(\|\Sigma^{1/2}x\|_2)^{-1}\,\Sigma x.$$

This is the network analogue of the per-edge statistical constraints of Section 7: the covariance enters through the cone term, while the quantile $F^{-1}(1-\gamma)$ plays the role of the yield parameter. The analogy is structural rather than a formal duality—the timing problem constrains tensions, whereas this problem constrains flows—and it is not used in the sequel.

5.5 Additional constraints and the barrier method

A constraint that is not a network constraint—for instance $s^{\mathsf{T}}x \le \gamma$ for a fixed vector s—breaks the network structure: the feasible set is no longer a network polytope, and (10) becomes a general convex program. Such a constraint is handled by a logarithmic *barrier* [34],

$$\phi(x) = -\log(\gamma - s^{\mathsf{T}}x), \qquad \nabla\phi(x) = \frac{s}{\gamma - s^{\mathsf{T}}x},$$

giving the approximation

$$\begin{array}{ll} \text{minimize} & f(x) + \frac{1}{t}\,\phi(x) \\ \text{subject to} & 0 \le x \le c, \quad A^{\mathsf{T}}x = b, \quad b(V) = 0, \end{array} \tag{11}$$

whose optimum tends to that of the original problem as $t \to \infty$. The *barrier method* alternates a *centering*

step—minimizing $tf + \phi$, by Newton's method in general—with an update $t \leftarrow \mu t$, $\mu > 1$, and stops when $1/t < \varepsilon$. The network-flow insight is that the centering step can itself be taken with a negative cycle: in place of the Newton direction one may use a negative cycle of the residual graph weighted by $\nabla(tf + \phi)$. The barrier method thus decomposes a constrained convex network program into a sequence of unconstrained cycle-cancellation steps.

6 Delay Padding

6.1 Motivation and formulation

When the TCG contains a negative cycle, no clock schedule can repair the circuit; the physical delays themselves must change. *Delay padding* inserts extra delay into selected data paths, usually by swapping a fast cell for a slower one. Its natural formulation adds a nonnegative padding vector p:

$$\text{find } p, u \quad \text{such that} \quad y \le d + p, \quad Au = y, \quad p \ge 0. \tag{12}$$

If the objective is to minimize the total inserted delay $\sum_e p_e$, the continuous problem has a compact dual. With multipliers $\lambda \ge 0$ for $y - p \le d$, μ for $Au - y = 0$, and $\nu \ge 0$ for $p \ge 0$, eliminating p, y, and u gives

$$\min \ \lambda^{\mathsf{T}} d \quad \text{s.t.} \quad 0 \le \lambda \le \mathbf{1}, \qquad A^{\mathsf{T}} \lambda = 0,$$

a minimum-cost circulation with unit capacities on the arcs of the timing constraint graph. The dual is therefore an ordinary network-flow problem, solvable by the network simplex method. This observation is appealing but, in modern designs, *impractical*: delay insertion is realized by cell substitution, so minimizing $\sum p$ does not reflect the true cost, and—more importantly— there may be no valid position to insert delay along some paths [35].

6.2 The structural configurations

A better strategy is to decide *where* delay may be inserted *before* deciding *how much* to insert. This is *path relationship analysis* (PRA). For a pair of registers i and j, the maximum-delay (setup) path and the minimum-delay (hold) path interact in one of the ways listed below, and each interaction determines how the TCG must be modified (Figure 6):

1. **Type (a), no insertion.** No cell on either path can be replaced (the path is already at the slow end of the library, or the nets are fixed); the TCG is unchanged.
2. **Type (b), independent.** The setup and hold paths are disjoint; delays p_s and p_h may be inserted independently, modeled by auxiliary nodes connected to the original vertices.
3. **Type (c), identical.** The two paths are the same set of cells, so a single delay must be shared: $p_s = p_h$.
4. **Type (d), containment.** The hold path is a sub-path of the setup path; padding the shared prefix affects both, so $p_s \ge p_h$.

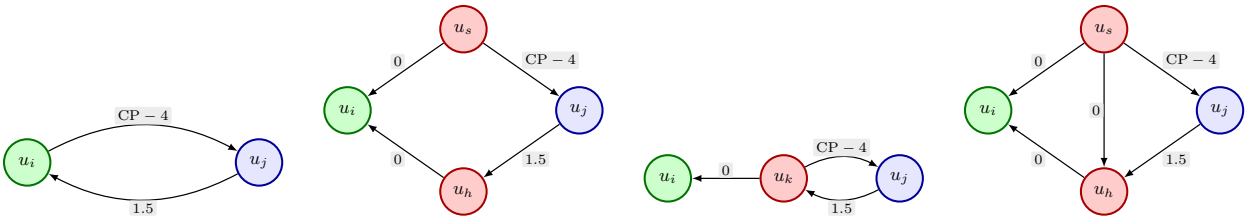
A fifth case, *partial overlap*—the two paths share a proper sub-path but neither contains the other—is common in practice and behaves like (d) on the shared segment and like (b) on the private segments. The four cases above are those that change the constraint graph in distinct ways; the padding formulation itself is developed in [35].

6.3 Delay padding as a modified feasibility problem

With the TCG augmented by auxiliary nodes and coupling edges, the question "can the timing be fixed by padding?" again becomes a feasibility problem on a modified graph: an infeasible instance returns a negative cycle, which certifies that padding alone is insufficient and that D_{ij} must be reduced or T_{CP} increased. In the modified graph the padding is a *potential* on the auxiliary edges, so the whole machinery of Section 3 applies unchanged. This "determine the position first, then the value" strategy is both more flexible and more faithful to physical design than the minimize-$\sum p$ flow formulation.

6.4 Discrete padding and its complexity

Formulation (12) treats the padding p as a continuous, nonnegative real vector. In a standard-cell flow this is an idealization: delay is inserted either by swapping a cell for a slower one from a discrete library or by adding whole buffer stages, so the true feasible set of p is *discrete* (and, per path, non-convex), and the exact problem is a mixed-integer linear program (MILP) rather than a network flow. The continuous relaxation studied here should therefore be read as (i) a lower bound on the achievable clock period and (ii) a guide to *which* paths are worth padding. Path relationship analysis narrows the search by pre-selecting physically realizable positions and by coupling the setup and hold paddings ($p_s = p_h$ or $p_s \ge p_h$), but it does not make the values continuous. Closing the gap between a relaxed padding and a discrete, cell-level solution is



(a) Type (a). (b) Type (b). (c) Type (c). (d) Type (d).

Figure 6: The four structural configurations for delay insertion. Type (a): no feasible insertion; (b): p_s, p_h independent; (c): $p_s = p_h$; (d): $p_s \geq p_h$.

a separate combinatorial problem; choosing positions *before* values, as above, is what keeps that second problem tractable in practice.

Discreteness does not by itself place the problem beyond convex machinery. Some discrete problems whose relaxations are integral—minimum-cost matching being the classic example—have polynomial-time algorithms built on separation, because a separating hyperplane can be found without enumerating the constraints [30]. Extending (12) to a discrete technology library therefore *may* be feasible, but it requires cell-level cost data and a proof that the resulting relaxation is integral; the continuous relaxation studied here is the first step.

7 Yield-Driven Scheduling under Process Variations

7.1 Timing yield

When process variations grow large enough, timing-failure-induced yield loss becomes a first-order concern, and the design objective shifts from the shortest nominal period to the *highest timing yield*. A circuit is called *functionally correct* for a given sample of process parameters if all of its setup- and hold-time constraints are satisfied; the *timing yield* is the fraction of manufactured samples that are correct [2], [17], [36], [37], [38].

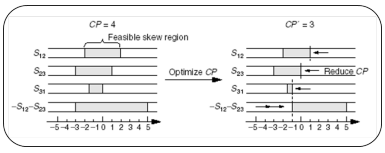


Figure 7: After period minimization, many skews sit on the boundary of their feasible skew regions; the narrow margins that remain are what process variations consume, which motivates yield-driven scheduling.

Maximizing yield is fundamentally different from minimizing a nominal period: it forces the schedule to reason about the *distributions* of the path delays rather than their worst-case values. Figure 7 illustrates why the margins left after period minimization are exactly the ones that variation consumes.

7.2 Primitive solutions and their shortcomings

Three elementary heuristics make the difficulty concrete.

1. **Margin pre-allocation.** Reserve a fixed timing margin Δd at both ends of every FSR,

$$\begin{aligned} &\underline{w}_{ij} \leq y_{ij} \leq \overline{w}_{ij} \\ \Longrightarrow \quad &\underline{w}_{ij} + \Delta d \leq y_{ij} \leq \overline{w}_{ij} - \Delta d \end{aligned}$$

   and then optimize the clock period. The margin is usually the maximum timing uncertainty, which is too pessimistic, and a single fixed Δd cannot reflect the different uncertainties of different data paths.
2. **Least center error square (LCES).** Since a skew near the center of its FSR leaves the most slack on both sides, one may place each y_{ij} as close as possible to the midpoint of $[\underline{w}_{ij}, \overline{w}_{ij}]$. Writing $m_{ij} = \overline{w}_{ij} - \underline{w}_{ij}$ for the width of the FSR and using lower and upper fractions $\ell_k, u_k \in [0, 0.5]$,

$$\underline{w}_{ij} + \ell_k\, m_{ij} \leq y_{ij} \leq \overline{w}_{ij} - u_k\, m_{ij},$$

$$\min \sum_k \left(\tfrac{1}{2} - \min(\ell_k, u_k)\right)^2,$$

   a small quadratic program [2], [17]. Minimizing the *total* center error, however, can drive individual slacks to zero, which is not optimal for yield.
3. **Incremental slack distribution.** Distribute slack greedily while checking all skew constraints

[39]; the method again ignores the differences between path delays.

All three fail for the same reason: they do not weight each FSR by the *variance* of the underlying path delays.

7.3 Statistical timing model

Under process variations the maximum and minimum path delays become random variables $\tilde{D}_{ij}$ and $\tilde{d}_{ij}$, and the setup- and hold-time constraints become probabilistic:

$$T_{\text{skew}}(i,j) \le T_{\text{CP}} - \tilde{D}_{ij} - T_{\text{setup}},$$

$$T_{\text{skew}}(i,j) \ge T_{\text{hold}} - \tilde{d}_{ij},$$

where $T_{\text{skew}}(i,j) = u_i - u_j$. After statistical static timing analysis (SSTA), every edge of the timing constraint graph carries a pair (μ_{ij}, σ_{ij})—the mean and standard deviation of its *arc weight*, equivalently of the corresponding slack—so the schedule must bound the *probability* of violation rather than its worst case (Figure 8). The arrival times u_i remain deterministic; only the arc weights are random. [40], [41], [42], [43], [44].

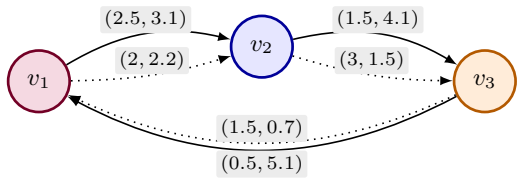


Figure 8: A statistical timing constraint graph: after SSTA each edge carries a pair (μ, σ)—mean and standard deviation—instead of a single deterministic weight.

7.4 EVEN: uniform slack maximization

The EVEN method assumes that all path delays have equal variance and maximizes a uniform margin β:

$$\max \beta \quad \text{s.t.} \quad u_i - u_j \le \mu_{ij} - \beta, \qquad u_j - u_i \le \mu^h_{ij} - \beta,$$

where μ_{ij} is the mean of the setup arc weight $T_{\text{CP}} - D_{ij} - T_{\text{setup}}$ and μ^h_{ij} the mean of the hold arc weight $d_{ij} - T_{\text{hold}}$. This is the *minimum mean cycle* problem, whose optimum is the mean edge weight of the critical cycle C,

$$\beta^\star = \min_C \frac{1}{|C|} \sum_{e \in C} \mu_e,$$

so the critical cycle is the *minimizer*, computable by Karp's algorithm or by the faster methods of Dasdan and Gupta [45], [46].

EVEN then proceeds iteratively, much like minimum balancing: it identifies the most timing-critical cycle, distributes the slack evenly along it, freezes the corresponding skews, replaces the cycle by a super-vertex, and repeats (Figure 9). For the resulting schedule the *setup* slack of each edge is

$$\text{slack}_{ij} = T_{\text{CP}} - D_{ij} - T_{\text{setup}} - \text{skew}_{ij},$$

with the hold slack defined analogously on the reverse arc. EVEN's weakness is exactly its assumption: the timing uncertainty of a long combinational path is normally larger than that of a short one, so an even slack distribution is not yield-optimal whenever the path delays along a critical cycle differ.

7.5 PROP: variance-proportional slack

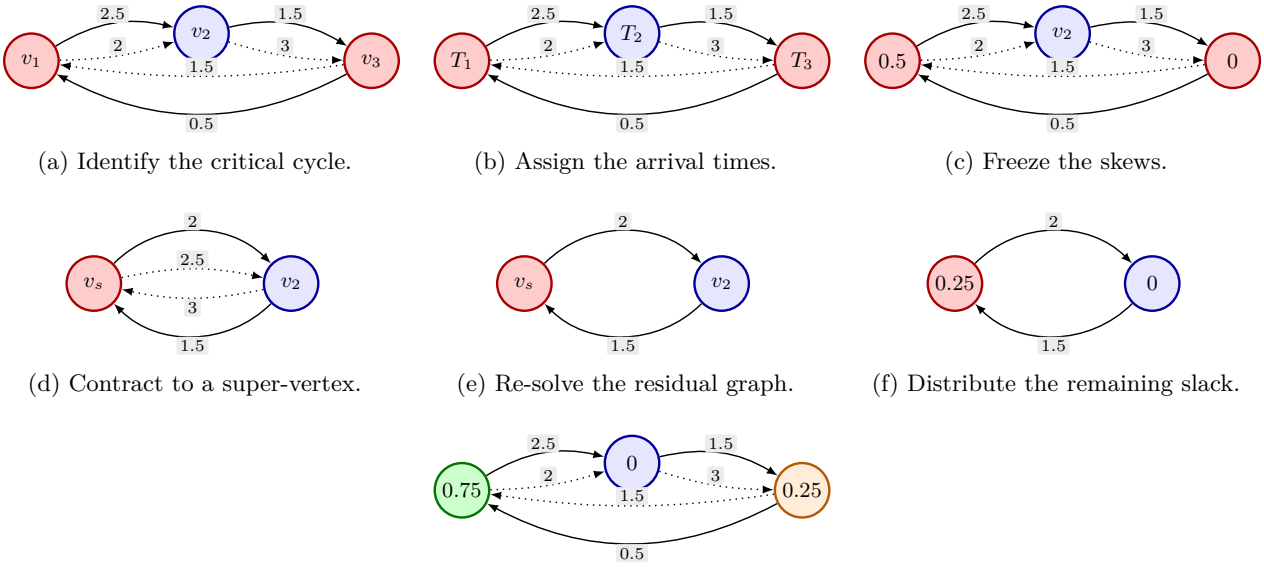
PROP models each gate delay as Gaussian with a common standard deviation σ and a path as the sum of n such gates, so a path delay is $\mathcal{N}(n\mu_0, n\sigma^2)$ and its standard deviation grows as $\sqrt{n}$. Measuring delays in units of a nominal gate delay—so that D_{ij} is approximately the stage count—PROP distributes the slack along the most critical cycle in proportion to those square roots by updating the TCG arc weights with a parameter α:

$$w^{\text{s}}_{ij}(\alpha) = T_{\text{CP}} - D_{ij} - T_{\text{setup}} - \alpha\,\sigma\sqrt{D_{ij}}, \qquad w^{\text{h}}_{ij}(\alpha) = d_{ij} - T_{\text{hold}} - \alpha\,\sigma\sqrt{}$$

For a fixed clock period, α is increased until Bellman–Ford reports infeasibility; the largest feasible α gives every edge of the most critical cycle a slack equal to its pre-allocated margin, after which the remaining skews are filled in by EVEN. PROP's weaknesses are its assumption of a common gate distribution, its use of path delay as a proxy for stage count, and the fact that the $\sqrt{D}$ weighting is a modelling choice rather than a consequence of the statistics.

7.6 FP-PROP: false-path awareness

A *false path* can never be sensitized and therefore never carries a signal (Figure 10). If false paths are treated as real, non-critical cycles are mistaken for critical ones, slack is diverted to them, and the genuine critical cycles receive too little margin; the overall timing yield then drops. FP-PROP therefore performs a sensitizable-critical-path search and removes false paths from the scheduling problem [36].



(a) Identify the critical cycle. (b) Assign the arrival times. (c) Freeze the skews.

(d) Contract to a super-vertex. (e) Re-solve the residual graph. (f) Distribute the remaining slack.

(g) The final schedule.

Figure 9: The iterative EVEN (minimum-balancing) process on a small timing constraint graph: identify the most critical cycle, distribute its slack evenly, freeze the arrival times, contract the cycle to a super-vertex, and repeat until a single vertex remains.

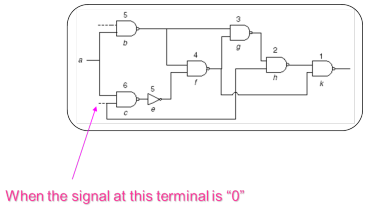


Figure 10: A false path: although a structural path exists, no input pattern sensitizes it, so it carries no signal and must be excluded from the timing constraints.

7.7 Most critical cycle and C-PROP

Traditionally the most critical cycle is the *minimum mean cycle*, weighted by $\sum \mu_{ij}/|C|$. Under unequal variances the criterion weights each edge by its own uncertainty, and the critical cycle is the one *minimizing* the ratio $\sum \mu_{ij} / \sum \sigma_{ij}$. C-PROP is exactly the corresponding slack-maximization problem,

$$\max \beta \quad \text{s.t.} \quad u_i - u_j \le \mu_{ij} - \sigma_{ij}\beta, \qquad u_j - u_i \le \mu^h_{ij} - \sigma^h_{ij}\beta,$$

a *minimum cost-to-time ratio cycle* problem whose optimum is

$$\beta^\star = \min_C \frac{\sum_{e\in C} \mu_e}{\sum_{e\in C} \sigma_e},$$

solved efficiently by Howard's algorithm [39]. C-PROP reduces to EVEN when all σ_{ij} are equal, and as a variance tends to zero it assigns only a minimal margin to that constraint while leaving more for the others—precisely the desired behavior. Figure 11 shows the optimum for a small example.

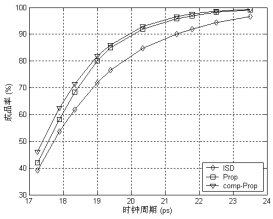


Figure 11: The optimum of the minimum cost-to-time ratio cycle for a C-PROP example.

7.8 Whole flow: super-vertex contraction

Once the arrival times on the most critical cycle are fixed, the cycle is replaced by a super-vertex v'. An in-edge (u, v) from an outside vertex u to a cycle member v becomes an in-edge (u, v') of mean $\mu(u, v) - T_v$, and an out-edge (v, u) becomes (v', u) of mean $\mu(v, u) + T_v$; the variance of the edge weight is unchanged, and parallel edges may remain. The process repeats until the graph is reduced to a single super-vertex or has no edges left. The final arrival time of a register is then the sum of the frozen arrival times along its path in the contraction tree, $T_i^{\text{final}} = \sum_v T_v$ (Figure 12).

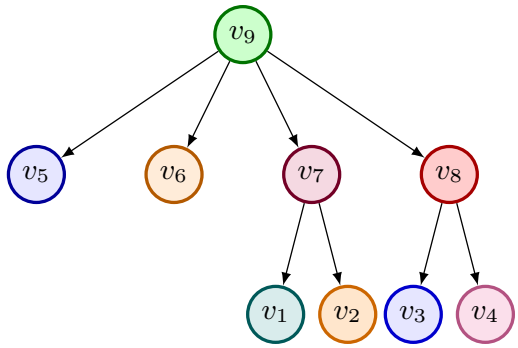


Figure 12: The contraction tree: each super-vertex merges a critical cycle, and the final arrival time of a register is the sum of the frozen arrival times along its root path—for example, $T_1^{\text{final}} = T_1 + T_7 + T_9$.

7.9 One parametric family

Minimum-period scheduling, EVEN, PROP, and C-PROP are four members of a single family of parametric potential problems, differing in the objective $g(\beta)$ and the right-hand sides $f_{ij}(\beta)$ (Table 1).

Here β is the clock period in the Min. CP row, a uniform time margin in the EVEN and PROP rows, and a multiple of the per-edge standard deviation in the C-PROP row; the units differ, and the table is a statement about form rather than about a common numerical scale. The Min. CP row is the one member whose right-hand sides *increase* with β: it is a minimization, and it fits the max template only after the substitution $\beta \mapsto -\beta$. For the other rows every f is non-increasing in β, so the feasible set is an interval and the optimum is its right endpoint.

7.10 Yield maximization and the correlation question

The statistical objectives above are instances of the max-min formulation

$$\max_{u} \; \min_{ij} \; \Pr\{u_j - u_i \le \tilde{W}_{ij}\},$$

where $\tilde{W}_{ij}$ is the random slack of arc (i, j). This is *not* exactly the timing yield, but it is easy to solve and—crucially—it needs no correlation information among the $\tilde{W}_{ij}$. Writing F_{ij} for the CDF of $\tilde{W}_{ij}$, it is equivalent to

$$\max \beta \quad \text{subject to} \quad u_j - u_i \le F_{ij}^{-1}(\beta), \qquad u_i - u_j \le F_{ji}^{-1}(\beta).$$

Because every CDF is monotone increasing, its inverse is well defined and the problem remains a parametric potential problem: $\max \beta$ subject to difference constraints whose right-hand sides decrease with β.

The reason no correlation is required is that the objective bounds each *individual* constraint separately, so only the *marginal* distributions are needed. This is not an assumption that the delays are uncorrelated: correlation is accounted for upstream, in the STA/SSTA step that produces $\tilde{D}_{ij}$ and $\tilde{d}_{ij}$, and once those marginals are fixed the per-edge objective needs no further correlation information. The price is that the result is a *surrogate* for, not the true, timing yield, which is a property of the *joint* distribution of the path delays; the surrogate is not monotone in any obvious way with respect to the joint yield, so a schedule that improves it can in principle degrade the true yield. Replacing the surrogate by the true joint yield would require the full correlation matrix, which contemporary SSTA does not report, and remains an open problem (Section 11). Figure 13 compares the schedules produced by the methods above.

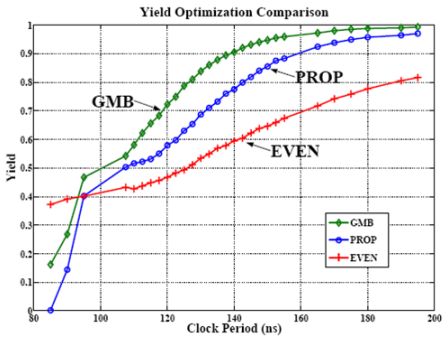


Figure 13: A comparison of the yield-driven scheduling methods on a common example.

7.11 Gaussian model and linearization

When the arc slacks are Gaussian, the probabilistic constraints can be written explicitly. With $z = \sqrt{2}\,\mathrm{erf}^{-1}(2\beta - 1)$ (the standard normal β-quantile), maximizing the yield parameter β becomes

$$\max \beta \quad \text{subject to} \quad u_i - u_j \le T_{\mathrm{CP}} - \mu_{ij}^D - \sigma_{ij}^D z, \qquad u_j - u_i \le \mu_{ij}^H - \sigma_{ij}^H z.$$

Because z is a strictly increasing function of β, maximizing β is equivalent to maximizing z, and in the variable z the constraints are affine. The result is a parametric potential problem with per-edge slopes σ_e; with T_{CP} fixed it is a max-ratio problem, solved by bisection on z.

7.12 Log-normal model

A log-normal model replaces the Gaussian quantile by an exponential one, giving

$$u_i - u_j \le T_{\mathrm{CP}} - \exp\!\big(\mu_{ij}^D + \sigma_{ij}^D z\big), \qquad u_j - u_i \le \exp\!\big(\mu_{ij}^H - \sigma_{ij}^H z\big).$$

Table 1: Clock skew scheduling as one parametric family. Every row is $\max\{g(\beta) : u_i - u_j \le f_{ij}(\beta),\ u_j - u_i \le f_{ji}(\beta)\}$ for the objective and bounds shown; the units of β differ between rows.

Problem	$g(\beta)$	$f_{ij}(\beta)$ (setup)	$f_{ji}(\beta)$ (hold)	unit of β
Min. CP	$-\beta$	$\beta - D_{ij} - T_{\text{setup}}$	$d_{ij} - T_{\text{hold}}$	time
EVEN	β	$T_{\text{CP}} - D_{ij} - T_{\text{setup}} - \beta$	$d_{ij} - T_{\text{hold}} - \beta$	time
PROP	β	$T_{\text{CP}} - D_{ij} - T_{\text{setup}} - \beta\sigma\sqrt{D_{ij}}$	$d_{ij} - T_{\text{hold}} - \beta\sigma\sqrt{d_{ij}}$	time
C-PROP	β	$T_{\text{CP}} - D_{ij} - T_{\text{setup}} - \sigma_{ij}\beta$	$d_{ij} - T_{\text{hold}} - \sigma^h_{ij}\beta$	number of σ

This avoids evaluating the error function. The problem is nonlinear and its feasible set is not convex, but feasibility is monotone in z, so bisection still applies; each test is a negative-cycle detection with transcendental weights, which makes this the most expensive member of the family.

7.13 Illustrative comparison

Figure 14 reproduces an illustrative comparison from the course notes [47]. It shows the qualitative behavior of the methods above on a small example; no benchmark, instance sizes, or runtime data accompany it, and it should not be read as an empirical comparison. Section 11.3 states why no such comparison is reported here.

		Even	*Prop*		*fp-Prop*	
Circuit	***CP***	**Yield (%)**	**Yield (%)**	**Imp. (%)**	**Yield (%)**	**Imp. (%)**
s1423	55.73	74.1	75.5	1.9	75.5	1.9
s1488	16.62	72.3	75.4	4.3	NA	NA
s1494	16.62	77.9	78.8	1.2	NA	NA
s5378	22.50	63.8	64.2	0.6	NA	NA
s9234.1	40.86	74.1	83.9	13.2	NA	NA
s13207.1	52.73	60.2	60.2	0.0	NA	NA
s35932	31.96	64.3	98.6	53.3	98.6	53.3
s38417	34.07	74.1	74.7	0.8	89.1	20.2
s38584.1	50.24	72.5	85.8	18.3	NA	NA
b04s	23.88	67.4	82.6	22.6	NA	NA
b05s	47.68	67.2	86.9	29.3	86.9	29.3
b06	4.92	63.1	75.1	19.0	NA	NA
b07s	26.23	74.8	72.7	−2.8	72.7	−2.8
b08	14.90	68.9	65.6	−4.8	65.6	−4.8
b09	7.68	62.4	78.3	25.5	NA	NA
b10	9.71	73.5	73.7	0.3	NA	NA
b11s	29.86	71.5	58.6	−18.0	80.9	13.1
b12	12.12	78.2	87.6	12.0	87.6	12.0

Figure 14: Illustrative comparison of the yield-driven scheduling methods; not a benchmark.

7.14 Co-optimization of clock period and yield

When the clock period and the yield are optimized jointly, the objective is the ratio

$$\min \frac{T_{\text{CP}}}{\beta} \quad \text{subject to} \quad y_{ij} \le T_{\text{CP}} - F_{ij}^{-1}(\beta)\ \ (i,j) \in E_s,$$

The setup bound tightens as the period shrinks and as the yield target rises; the hold bound tightens as the yield target rises, because $F_{ij}^{-1}(1-\beta)$ falls with β. Both involve F^{-1}, which is not concave over $[0,1]$: for a Gaussian it is convex above the median and concave below, and the two constraints therefore require opposite curvatures. They are compatible exactly when β and $1-\beta$ lie on opposite sides of the median—for Gaussians, when $\beta > 1/2$. Restricting to $\beta \ge 0.8$ is therefore sufficient for the Gaussian model, but the corresponding threshold for a log-normal or GEV marginal must be computed from its mode, and the claim of convexity is distribution-dependent.

For a trial value of γ, the constraint $T_{\text{CP}} \le \gamma\beta$ is linear, and the remaining feasibility test is a parametric potential problem; bisection on γ converges to the optimal ratio. The problem is therefore solved as a sequence of convex feasibility problems,

$$\text{find } (y, T_{\text{CP}}, \beta) \quad \text{s.t.} \quad T_{\text{CP}} - \gamma\beta \le 0, \quad y_{ij} \le T_{\text{CP}} - F_{ij}^{-1}(\beta), \quad y_{ji} \le F_{ij}^{-1}$$

rather than as a single convex program. The constraint $T_{\text{CP}} - \gamma\beta \le 0$ is bilinear in (γ, β) and is *not* convex when γ is a variable, which is why the ratio is handled by bisection rather than by an epigraph variable.

7.15 Yield-driven delay padding and its dual

The padding formulation of Section 6 and the yield formulation of the present section can be merged into a single program that trades yield against the cost of inserted delay. With a padding vector $p \ge 0$, a weight γ on the yield, per-edge Gaussian delays of mean d_{ij} and standard deviation σ_{ij}, and a padding-cost vector k, the problem is

$$\max\ \gamma\beta - k^{\mathsf{T}}p \quad \text{subject to} \quad \beta \le \Pr\{y_{ij} \le \mathbf{d}_{ij} + p_{ij}\}, \quad Au = y, \quad p \ge 0$$
$$y_{ji} \le F_{ij}^{-1}(1-\beta)\ \ (j,i) \in E_h, \quad T_{\text{CP}} \ge 0,\ \ 0 \le \beta \le 1.$$

If the quantile is linearized as $Q(\beta) = d - \beta\sigma$—equivalently, if β is reparameterized by its Gaussian z-score—the chance constraint becomes the linear

constraint $y \le d - \beta\sigma + p$, and the program is the linear program

$$\max\ \gamma\beta - k^{\mathsf{T}}p \quad \text{subject to} \quad y \le d - \beta\sigma + p, \quad Au = y, \quad p \ge 0$$

Its dual is a min-cost circulation with two side constraints,

$$\min\ \lambda^{\mathsf{T}}d \quad \text{subject to} \quad 0 \le \lambda \le k, \quad A^{\mathsf{T}}\lambda = 0, \quad \sigma^{\mathsf{T}}\lambda = \gamma,$$

in which the last constraint is the non-network one handled by the barrier method (11). Sections 6 and 7 are therefore primal and dual views of a single problem, which is why they share the same negative-cycle core. The linearization is a modelling choice: with the exact Gaussian quantile the program is no longer an LP, and the dual acquires a nonlinear term.



Figure 15: Path-delay distributions are not Gaussian: they are asymmetric and heavy-tailed, so a Gaussian fit captures the bulk but understates the tails that govern setup and hold failures (illustration).

8 Non-Gaussian and Heavy-Tailed Delay Models

8.1 Why the Gaussian model fails

At 65,nm and below, the distribution of a *maximum* path delay is typically asymmetric and heavy-tailed [42], [48], so the central limit theorem does not rescue the Gaussian assumption. Setup failures occur in slow corners and depend on the upper tail; hold failures occur in fast corners and depend on the lower tail. A model that fits the bulk but misses the tails therefore mis-estimates yield precisely where it matters (Figure 15, reproduced from the course notes [47]).

8.2 Unimodality and quantile functions

A continuous distribution with mode m is *unimodal* if its density increases up to m and decreases after it, so its CDF is convex for $x < m$ and concave for $x > m$. Normal, log-normal, and log-logistic distributions are unimodal, and their *quantile functions* are available in closed form:

$$\begin{aligned} &\text{Normal:} && \mu + \sigma\sqrt{2}\,\mathrm{erf}^{-1}(2p-1), \\ &\text{Log-normal:} && \exp\!\big(\mu + \sigma\sqrt{2}\,\mathrm{erf}^{-1}(2p-1)\big), \\ &\text{Log-logistic:} && \alpha\big(\tfrac{p}{1-p}\big)^{1/\kappa}. \end{aligned}$$

For the log-normal distribution the mode is $\exp(\mu-\sigma^2)$ and the CDF at the mode is $\frac{1}{2}\big(1+\mathrm{erf}(-\sigma/\sqrt{2})\big)$. Because the CDF is convex below the mode and concave above it, the quantile function $Q = F^{-1}$ is *concave* on $(0, F(m))$ and *convex* on $(F(m), 1)$. This is the property used in Section 7 when the co-optimization program is claimed to be convex on a high-yield range.

8.3 Generalized extreme value distribution

The *generalized extreme value* (GEV) distribution [49], [50] unifies the three extreme-value types and models skewness and tail weight through three parameters: location μ, scale $\sigma > 0$, and shape ξ. Its PDF and CDF are

$$f(x) = \frac{1}{\sigma}\,t(x)^{\xi+1}e^{-t(x)}, \qquad F(x) = e^{-t(x)},$$
$$t(x) = \Big[1 + \xi\,\frac{x-\mu}{\sigma}\Big]^{-1/\xi} \quad (\xi \neq 0).$$

The quantile function is

$$Q(\beta) = \mu + \frac{\sigma}{\xi}\Big((-\ln\beta)^{-\xi} - 1\Big). \tag{13}$$

Because circuit delays are strictly positive and their maxima are heavy-tailed, the case $\xi > 0$ (Fr'echet) is the relevant one. Two cautions apply. First, (13) is the quantile of a *maximum*; applying the same family to a minimum requires the reflected variable, and the tail index of a minimum is the negative of that of the maximum, so a single ξ cannot describe both tails unless the two distributions are reflections of each

other. Second, the GEV support is $1+\xi(x-\mu)/\sigma > 0$: for $\xi > 0$ the support is bounded below by $\mu - \sigma/\xi$, which may be negative, whereas delays are positive; for $\xi < 0$ the upper tail is bounded, which contradicts the heavy-tail premise. In practice the distribution must be truncated at zero and the fit checked against the data, not assumed.

8.4 Statistical timing constraints with GEV

With GEV models for the maximum and minimum path delays, the setup and hold constraints of a critical path with skew T_{skew} become probabilistic,

$$T_{\text{skew}} \le T_{\text{CP}} - \tilde{D} - T_{\text{setup}}, \qquad T_{\text{skew}} \ge T_{\text{hold}} - \tilde{d},$$

and are converted to their deterministic equivalents through the quantile function (13):

$$T_{\text{CP}} - T_{\text{setup}} - T_{\text{skew}} \ge Q_D(\beta),$$
$$T_{\text{hold}} - T_{\text{skew}} \le Q_d(1-\beta).$$

The setup constraint is sensitive to the *upper* tail $Q_D(\beta)$ (slow corners), whereas the hold constraint is sensitive to the *lower* tail $Q_d(1-\beta)$ (fast corners). Because every CDF is monotone increasing, Q is monotone, so the yield parameter β remains a single scalar and the parametric framework of Section 4 still applies. In general Lawler's binary search solves the resulting problem; for the special GEV form one may bypass the explicit quantile by searching directly on a transformed parameter, exactly as in the Gaussian linearization of Section 7. The GEV model has been used for yield-driven skew scheduling by Wang et al. [18] and, in the form above, by Zhu et al. [19].

9 Algorithms

9.1 Overview

Table 2 compares the algorithms used in this survey. They share one primitive: the detection and cancellation of negative cycles. No single row dominates. The clean parametric bounds require unit slopes; the general min-ratio problem needs a different parametric-search algorithm; and Howard's practical speed comes without a polynomial guarantee. The hypothesis column is therefore part of the result, not a footnote.

9.2 Lawler's binary search

Lawler's method brackets β and halves the interval with a feasibility test (Algorithm 1) at each step. It is easy to implement and robust to nonlinear $d(\beta)$, at the cost of repeated full-graph scans; each test costs $O(nm)$ and the bracket shrinks by a factor of two per test (Figure 16).

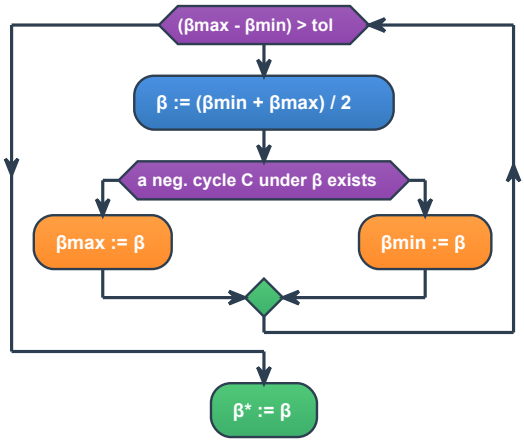


Figure 16: Lawler's binary-search algorithm: bisect the bracket $[\beta_{\min}, \beta_{\max}]$ and test the timing constraint graph for a negative cycle; the upper or lower bound is updated until the tolerance is met.

9.3 Howard's policy iteration

Howard's algorithm is the method of choice for the single-parameter problem in practice. It maintains one chosen outgoing arc per vertex (a *policy*); the policy graph contains a cycle, whose current ratio gives an improved β; the potentials are then recomputed with respect to the parameterized weights and the policy is updated. Each iteration produces a strictly better β, and the algorithm returns the optimum together with *one* critical cycle. The gains of the intermediate policies are monotone, but those intermediate cycles are not themselves critical, and no polynomial worst-case bound is known; the known lower bounds for the deterministic problem are quadratic [51], while exponential lower bounds exist for the more general stochastic setting. Variants that interleave binary-search pivots converge even faster, and hybrid Lawler–Howard methods are common in practice (Figures 17 and 18).

9.4 Karp's minimum mean cycle

For the equal-variance case, the minimum mean cycle admits a closed-form characterization [45], computable in $O(nm)$ time from shortest-path distances

Table 2: Algorithms for the problems used in this survey. Here $n = |V|$, $m = |E|$, K bounds the integer edge slopes, and ϵ is the binary-search tolerance. The hypothesis column is part of the result.

Problem	Method	Hypothesis	Cost / behavior
Min mean cycle	Karp	—	$O(nm)$
Min ratio cycle	Lawler	finite range, tolerance ϵ	$O(nm \log(1/\epsilon))$
Min ratio cycle	Megiddo	positive times	$O(n^2 m \log n)$
Min ratio cycle	Bringmann et al.	positive times	$\tilde{O}(m^{3/4} n^{3/2})$
Parametric shortest path	Karp–Orlin	slopes $0/-1$	$O(nm \log n)$
Parametric shortest path	Young–Tarjan–Orlin	slopes $0/-1$	$O(nm + n^2 \log n)$
Parametric shortest path	Young–Tarjan–Orlin	slopes in $[-K, 0]$	$O(Knm + Kn^2 \log n)$
Min mean / ratio cycle	Howard	—	exact; no polynomial bound

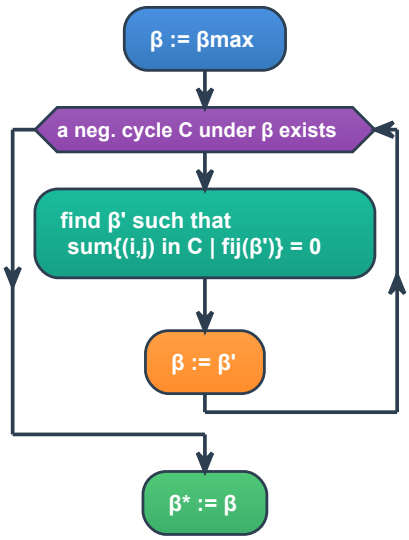


Figure 17: Howard's policy-iteration algorithm: a cycle of the current policy yields an improved β, after which a new policy is computed from the parameterized shortest paths.

with a bounded number of edges; the computation is n Bellman–Ford passes, one per path length. Karp's algorithm is the canonical subroutine for EVEN and for the minimum-mean-cycle form of any scheduling instance. The broader family of minimum-ratio algorithms is surveyed by Dasdan and Gupta [46] and Dasdan [28]; the parametric approach originates with Karp and Orlin [15] and Gusfield [52], and Carstensen [53] showed that arbitrary slopes can produce $n^{\Omega(\log n)}$ breakpoints, which is why the unit-slope hypothesis in Table 2 cannot be dropped. The current best strongly polynomial bound for the general min-ratio problem is due to Bringmann, Hansen, and Krinninger [54].

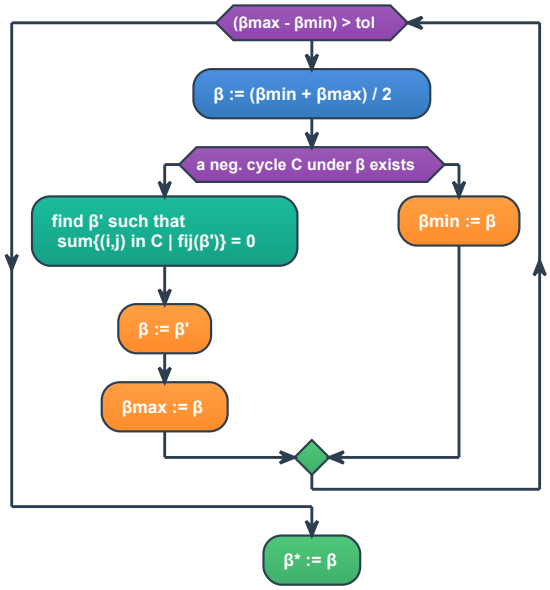
9.5 From schedule to a realizable clock tree

The scheduling algorithms return arrival times, but these must be *realized* by a physical clock tree. Here the network viewpoint again pays off: the minimum balancing algorithm not only produces a schedule but also a contraction tree that encodes the order of criticality (Figure 5). Section 10 discusses how to exploit this structure.

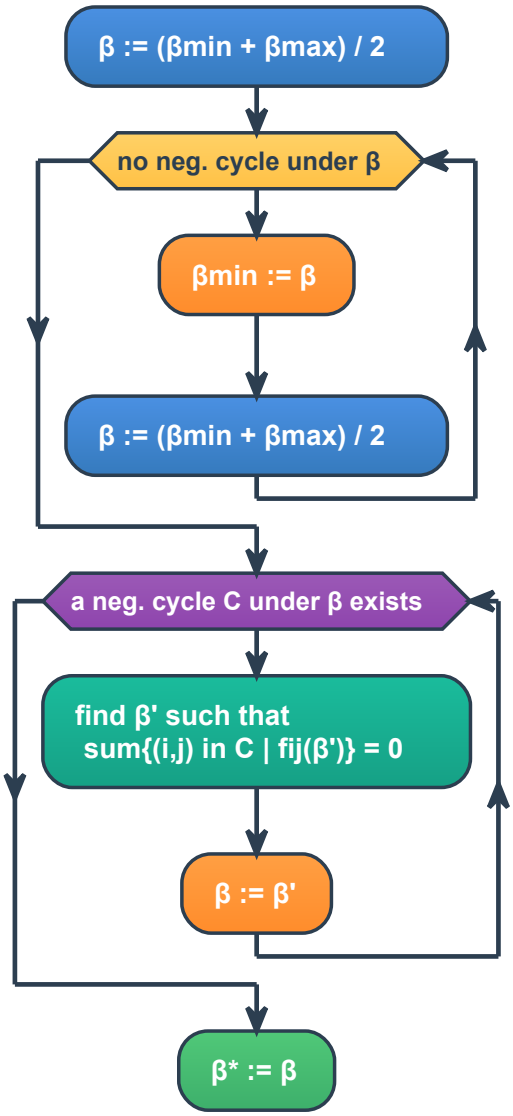
10 Clock-Tree Synthesis and Co-optimization

10.1 Realizing a skew schedule

A clock tree delivering prescribed arrival times is built by *deferred-merge embedding* (DME) with Elmore delay and buffer insertion. Constraining the tree to deliver a *bounded* skew is possible but significantly complicates the algorithm. The schedule itself should carry the matching bound: the program of Section 4 constrains only differences $u_i - u_j$, so it must be augmented with $0 \le u_i \le U_{\max}$ (equivalently, $\max_i u_i - \min_i u_i \le U_{\max}$) to exclude schedules whose



(a) Hybrid.



(b) Improved Howard.

Figure 18: Hybrid and improved variants interleave binary-search pivots with policy iteration, accelerating convergence.

spread no tree can deliver; without such a bound the optimum may place two registers a full period apart. The practical recommendation is to keep the scheduled skews as a budget: if the schedule is over-optimized, the tree becomes hard to build, and budgeting techniques must be applied [4]. After construction, a detailed timing analysis (not merely Elmore delay) should confirm the arrival times.

Realizing a prescribed skew is not free. Deferred-merge embedding places merging points and inserts buffers to meet target delays, and the buffer count, wirelength, and via count all grow with the *spread* of the required arrival times. Large skews between physically distant registers are the expensive case: they force detours in the clock wiring, raise clock-tree *power*, consume routing resources, and can induce *congestion* that ripples back into signal routing. A schedule that is optimal on the abstract TCG may therefore be unattractive, or even unrealizable, once these costs are counted. This is the central caveat of the approach: the algorithms here optimize a *delay-only* objective, and a production flow must re-optimize with the physical clock tree, or budget the skews so that the tree stays cheap to build.

10.2 Target skew versus actual skew

What the scheduler produces is a statement about *target* skews, and it must not be confused with the skew the silicon actually exhibits. The two concepts are distinct:

- the *target skew* is the value the scheduler intends to realize—the deterministic potential difference $y^\star_{ij} = u_i - u_j$ returned by the optimization; and
- the *actual skew* is the value the built clock tree delivers, which is a *random variable*, because the buffers and wires that produce it are subject to process variation.

Writing the realized arrival time at register i as $u_i + \delta_i$, where δ_i is the random clock-arrival perturbation, the actual skew of path $i \to j$ is

$$y_{ij} = (u_i + \delta_i) - (u_j + \delta_j) = y^\star_{ij} + (\delta_i - \delta_j).$$

The target skew is thus a deterministic *design intent*—we schedule a meeting at 10:00, not at 10:00 $\pm$ 34 minutes—whereas the actual skew is a stochastic *outcome* whose spread is governed by the correlation of the perturbations δ_i and δ_j. Two consequences follow. First, the scheduled skews are best treated as a *budget* rather than an exact requirement, since a tree that chases an over-optimized target becomes expensive and may still miss it, as noted above. Second, yield cannot be judged from the target skews alone; it requires a model of the actual skew as a random variable, which is exactly what the statistical methods of Section 7 supply. The target/actual distinction is therefore the interface between scheduling and clock-tree synthesis: the scheduler proposes a plan, and the tree—together with the variation it carries—determines what is realized.

The direction of causation reinforces the distinction. The schedule is computed *before* the clock tree exists, so it cannot be defined in terms of the skew that tree will eventually produce; the scheduler emits a target, and CTS then tries to approach it. The uncertainty of the underlying delays is already carried by the maximum and minimum path delays $\overline{w}$ and $\underline{w}$ supplied by STA/SSTA, so the target vector itself must remain a single, deterministic value, much as a meeting is scheduled for 03:25 p.m. and not for a distribution of starting times. The distinction is therefore one of role rather than of statistics: the scheduler emits a deterministic target, while the timing engine supplies the distributions from which that target is computed. An arrival time reported after CTS is an *actual* skew, not the quantity optimized in Sections 4–7.

10.3 Criticality-ordered topology and placement

A useful heuristic is to make the clock-tree topology follow the criticality hierarchy: registers that belong to the same contracted cycle should share a branch. Two benefits follow. First, a lower branch has smaller skew *variation*, so the most critical registers—placed deepest—enjoy the strongest correlation and the largest cancellation of variation. Second, if the placer also follows the topology—placing registers of the same branch physically together—spatial correlation further reduces skew variation. Since contemporary statistical timing analysis does not report the correlation information needed for a first-principles optimization, aligning topology, placement, and criticality is a valuable, practical surrogate [4], [5].

10.4 Co-optimization after the tree is built

After clock-tree synthesis the delay estimates are more accurate, raising the question of whether to re-schedule. Some methods relax the constraint to $1.2\,u_i - 0.8\,u_j \le w_{ij}$, reflecting that a built tree cannot realize an exact skew. The coefficients differ from unity, so the constraint is no longer a difference con-

straint and the negative-cycle test does not apply; the resulting problem remains a linear program and is solved by a general LP method. It is also possible to relax the fixed arrival time to an *interval*, which reduces clock-tree wirelength and makes the schedule robust to unavoidable process variation.

11 Discussion and Open Problems

11.1 What the network formulation buys

Framing clock skew scheduling as a parametric potential problem yields three concrete benefits. First, *efficiency*: the combinatorial algorithms are faster than generic linear programming, and the parametric shortest-path methods are faster still. Second, *certificates*: an infeasible instance returns the most critical cycle, which is directly actionable, and a feasible schedule comes with a criticality hierarchy that guides clock-tree synthesis. Third, *extensibility*: delay padding and the yield-driven variants are all obtained by modifying the graph rather than the algorithm.

11.2 Reference implementation

The framework of this survey is realized by two small companion libraries, released as sibling packages in Python, C++, and Rust [55], [56], sharing one structure so that the algorithmic core stays portable. The code is not a contribution of this article—the survey's claims remain as stated in Section 11.3—but it makes the reductions concrete and reproducible.

The combinatorial core lives in `digraphx`. It provides Howard's policy iteration, the minimum cost-to-time ratio cycle, and the parametric potential solver of Sections 4 and 9. A single clock-skew-scheduling entry point takes the objective as a pair of `(cost, time)` columns, so the four rows of the family table (Table 1) are thin adapters over one code path; the minimum-period row is handled by a negated-column transform that turns its maximum-cycle-ratio problem into the same minimum-ratio problem. Because the parameter is external to the graph, one routine serves both the exact combinatorial solvers and the ellipsoid oracle.

The separation oracles live in `netoptim`. `NetworkOracle` wraps the negative-cycle finder and, on detecting a negative cycle, returns the cutting plane of Section 4.4; `OptScalingOracle` and `ClockScheduleOracle` package the matrix-scaling and two-parameter (period, uniform-margin) problems as further instances of the same oracle contract. The two-parameter oracle of `ClockScheduleOracle` is the EVEN-style problem of Section 7, in which β is a time margin; reaching the probability-level yield model would replace β by the corresponding quantile and keep the same cut construction. The `ellalgo` libraries supply the ellipsoid and cutting-plane drivers that consume these planes.

Two implementation points deserve recording, because both are easy to get wrong and both concern the certificate rather than the arithmetic. First, the oracle must emit a *valid* separating plane. Howard's per-edge verification can surface a cycle whose total weight is not actually negative; the cycle generator in `digraphx` filters such cycles, and the oracle in `netoptim` retains the most-violated cycle it is given. Second, the actionable output is the *most critical* cycle, so the feasibility oracle selects the minimum-mean negative cycle rather than the first one returned—the certificate discussed in Section 3.

11.3 Scope and limitations

This article is a survey, not a report of new experiments. Its claims are of two kinds: mathematical statements—the reductions to the parametric potential problem and the negative-cycle criterion—which are exact under the stated model; and practical claims (efficiency, robustness), which are inherited from the cited literature and are not re-validated here.

The model makes four idealizations that a production flow must confront. (i) *Physical realizability*: arrival times are free continuous potentials, whereas a real clock tree incurs power, area, buffer, and congestion costs that grow with skew spread (Section 10). (ii) *Discrete delay*: padding is quantized by cell substitution or buffer insertion, making the exact problem a MILP, of which the continuous network-flow formulation is a relaxation (Section 6). (iii) *Per-edge statistics*: the yield formulations use per-edge marginals and are not the true, joint timing yield; spatial correlation is accounted for upstream, in STA/SSTA, rather than in the scheduler (Section 7). (iv) *Flatness*: the difference-constraint model assumes one synchronous graph, whereas hierarchical timing abstractions, clock-domain crossings, and dynamic clock gating break that assumption. The framework is thus best understood as the exact *algorithmic core* of a larger physical-optimization problem, to which the surrounding engineering steps add costs and constraints that the core itself does not capture.

No new experiments are reported. That is a limitation of scope, not a claim that experiments are impossible:

random timing constraint graphs, the public ISCAS-89 and ITC-99 suites, an open PDK, and an open RTL-to-GDS flow all support a controlled comparison, and the algorithms themselves are small. What such a study would not settle is the physical side—whether the scheduled skews survive clock-tree construction and silicon—which does require measurement data. The open problems below are stated so that groups with that access can pursue them.

11.4 Modelling assumptions

The formulation rests on assumptions that are easy to misread; they are stated here once, positively.

- **The optimized arrival times are deterministic targets.** They are computed before the clock tree exists and are not random variables. The random quantity is the *actual* skew the tree delivers (Section 10); the statistics enter through the path-delay bounds, not through the target vector.
- **The yield objective is per-edge.** It consumes marginal path-delay distributions and does not require the joint distribution. Correlation is accounted for upstream, when SSTA produces those marginals; the objective is a surrogate for the joint yield, not the joint yield itself (Section 7).
- **The schedule precedes the physical stages.** Power, area, and congestion are not in the objective. The schedule is a budget that the physical flow must realize or revise (Section 10).
- **The graph is flat.** Hierarchy, clock-domain crossings, and gating are boundary conditions supplied by the timing engine, not properties of the scheduling core (Section 11.3).
- **No benchmark is reported.** The empirical claims are inherited from the cited literature (Section 11.3).

11.5 Open problems

Several questions remain open.

- **Correlation-aware yield.** Incorporating spatial correlation into a *joint* yield objective—as opposed to the per-edge surrogate used here—remains open, although correlation itself is well modeled in SSTA [40], [41].
- **Quantized delay padding.** Delay insertion by cell swapping is inherently quantized; integer or mixed-integer versions of (12) deserve more attention.
- **Non-Gaussian, multivariate yield.** Fitting GEV marginals is only half the story; the joint distribution and its tails govern the true yield.
- **Co-optimization.** Joint placement, clock-tree synthesis, and skew scheduling under a single objective is still largely unsolved.
- **Physical cost of skew.** The formulations optimize arrival times without charging for clock-tree power, area, buffer count, or routing congestion; a cost-aware objective that prices large, physically distant skews is missing.
- **Hierarchical and gated designs.** Flat difference constraints assume a single synchronous graph; hierarchical timing, clock-domain crossings (CDCs), and dynamic clock gating call for decomposed or interface-constrained formulations. Multi-domain scheduling is the first step [57].
- **Independent evaluation.** The speedups quoted in this survey are those reported in the cited papers; an apples-to-apples study on modern multi-million-gate benchmarks remains to be done.

12 Conclusion

Clock skew scheduling turns the inevitable, unwanted skew of a clock network into a deliberate design variable. This survey has argued that the problem has a single mathematical core: the timing constraints are a system of difference constraints, feasibility is the absence of a negative cycle in the timing constraint graph, and the optimization variants—minimum clock period, maximum slack, maximum yield—are parametric shortest-path problems with fast combinatorial solvers. Building on this core, delay padding, yield-driven scheduling, and heavy-tailed GEV delay models extend the framework without changing its algorithmic heart. The critical cycle and the criticality hierarchy returned by these solvers double as actionable design guidance for clock-tree synthesis and placement. As technology scales and process variation dominates, these ideas—useful skew, negative-cycle analysis, and parametric optimization—will remain central to timing closure.

References

[1] J. P. Fishburn, "Clock skew optimization," *IEEE Transactions on Computers*, vol. 39, no. 7, pp. 945–951, 1990.

[2] J. L. Neves and E. G. Friedman, "Optimal clock skew scheduling tolerant to process variations," in *Proceedings of the 33rd annual design automation conference (DAC)*, 1996, pp. 623–628. doi: 10.1145/240518.240636.

[3] I. S. Kourtev and E. G. Friedman, *Timing optimization through clock-skew scheduling*. Boston, MA: Springer, 2000.

[4] S. Held, B. Korte, J. Mäßberg, M. Ringe, and J. Vygen, "Clock scheduling and clock tree construction for high performance ASICs," in *Proceedings of the IEEE/ACM international conference on computer-aided design (ICCAD)*, 2003, pp. 232–239.

[5] K. Wang, L. Duan, and X. Cheng, "ExtensiveSlackBalance: An approach to make front-end tools aware of clock skew scheduling," in *Proceedings of the 43rd annual design automation conference (DAC)*, 2006, pp. 951–954. doi: 10.1145/1146909.1147150.

[6] C. E. Leiserson and J. B. Saxe, "Optimizing synchronous systems," in *Proceedings of the 22nd annual symposium on foundations of computer science (FOCS)*, 1981, pp. 23–36.

[7] C. E. Leiserson and J. B. Saxe, "Retiming synchronous circuitry," *Algorithmica*, vol. 6, no. 1, pp. 5–35, 1991.

[8] S. M. Burns, "Performance analysis and optimization of asynchronous circuits," PhD thesis, California Institute of Technology, 1991.

[9] K. A. Sakallah, T. N. Mudge, and O. A. Olukotun, "Analysis and design of latch-controlled synchronous digital circuits," *IEEE Transactions on Computer-Aided Design*, vol. 11, no. 3, pp. 322–333, 1992.

[10] N. V. Shenoy, R. K. Brayton, and A. L. Sangiovanni-Vincentelli, "Graph algorithms for clock schedule optimization," in *Proceedings of the IEEE/ACM international conference on computer-aided design (ICCAD)*, 1992, pp. 132–136.

[11] S. S. Sapatnekar and R. B. Deokar, "Utilizing the retiming-skew equivalence in a practical algorithm for retiming large circuits," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 15, no. 10, pp. 1237–1248, 1996.

[12] Y. Liu, M. C. Papaefthymiou, and E. G. Friedman, "Maximizing performance by retiming and clock skew scheduling," in *Proceedings of the 36th annual design automation conference (DAC)*, 1999, pp. 231–236.

[13] E. L. Lawler, *Combinatorial optimization: Networks and matroids*. New York: Holt, Rinehart; Winston, 1976.

[14] N. Megiddo, "Combinatorial optimization with rational objective functions," *Mathematics of Operations Research*, vol. 4, no. 4, pp. 414–424, 1979.

[15] R. M. Karp and J. B. Orlin, "Parametric shortest path algorithms with an application to cyclic staffing," *Discrete Applied Mathematics*, vol. 3, no. 1, pp. 37–45, 1981.

[16] N. E. Young, R. E. Tarjan, and J. B. Orlin, "Faster parametric shortest path and minimum-balance algorithms," *Networks*, vol. 21, no. 2, pp. 205–221, 1991.

[17] I. S. Kourtev and E. G. Friedman, "Clock skew scheduling for improved reliability via quadratic programming," in *Proceedings of the IEEE/ACM international conference on computer-aided design (ICCAD)*, 1999, pp. 239–243. doi: 10.1109/ICCAD.1999.810655.

[18] Y. Wang *et al.*, "Timing yield driven clock skew scheduling considering non-Gaussian distributions of critical path delays," in *Proceedings of the 45th annual design automation conference (DAC)*, 2008, pp. 223–226.

[19] K. Zhu, W.-S. Luk, and L. Wang, "Yield-driven clock skew scheduling based on generalized extreme value distribution," in *Proceedings of the 30th asia and south pacific design automation conference (ASP-DAC)*, 2025, pp. 425–432. doi: 10.1145/3658617.3697739.

[20] A. J. Hoffman, "Some recent applications of the theory of linear inequalities to extremal combinatorial analysis," in *Combinatorial analysis*, vol. 10, in Proceedings of symposia in applied mathematics, vol. 10., American Mathematical Society, 1960, pp. 113–127.

[21] V. R. Pratt, "A new approach to the analysis of timing," in *Proceedings of the 14th design automation conference (DAC)*, 1977, pp. 308–312.

[22] R. E. Shostak, "Deciding linear inequalities by computing loop residues," *Journal of the ACM*, vol. 28, no. 4, pp. 769–779, 1981.

[23] R. Dechter, I. Meiri, and J. Pearl, "Temporal constraint networks," *Artificial Intelligence*, vol. 49, no. 1–3, pp. 61–95, 1991.

[24] R. Bellman, "On a routing problem," *Quarterly of Applied Mathematics*, vol. 16, no. 1, pp. 87–90, 1958.

[25] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.

[26] B. V. Cherkassky and A. V. Goldberg, "Negative-cycle detection algorithms," *Mathematical Programming*, vol. 85, no. 2, pp. 277–311, 1999.

[27] R. A. Howard, *Dynamic programming and markov processes*. Cambridge, MA: MIT Press, 1960.

[28] A. Dasdan, "Experimental analysis of the fastest optimum cycle ratio and mean algorithms," *ACM Transactions on Design Automation of Electronic Systems*, vol. 9, no. 4, pp. 385–418, 2004.

[29] L. G. Khachiyan, "A polynomial algorithm in linear programming," *Soviet Mathematics Doklady*, vol. 20, pp. 191–194, 1979.

[30] M. Grötschel, L. Lovász, and A. Schrijver, *Geometric algorithms and combinatorial optimization*. Berlin: Springer, 1988.

[31] X. Zhou, W.-S. Luk, H. Zhou, F. Yang, C. Yan, and X. Zeng, "Multi-parameter clock skew scheduling," *Integration, the VLSI Journal*, vol. 48, pp. 129–137, 2015, doi: 10.1016/j.vlsi.2014.07.005.

[32] C. Albrecht, B. Korte, J. Schietke, and J. Vygen, "Cycle time and slack optimization for VLSI-chips," in *Proceedings of the IEEE/ACM international conference on computer-aided design (ICCAD)*, 1999, pp. 232–238. doi: 10.1109/ICCAD.1999.810654.

[33] R. K. Ahuja, T. L. Magnanti, and J. B. Orlin, *Network flows: Theory, algorithms, and applications*. Englewood Cliffs, NJ: Prentice Hall, 1993.

[34] S. Boyd and L. Vandenberghe, *Convex optimization*. Cambridge, UK: Cambridge University Press, 2004.

[35] N. V. Shenoy, R. K. Brayton, and A. L. Sangiovanni-Vincentelli, "Minimum padding to satisfy short path constraints," in *Proceedings of the IEEE/ACM international conference on computer-aided design (ICCAD)*, 1993, pp. 156–161.

[36] J.-L. Tsai, D. H. Baik, C. C.-P. Chen, and K. K. Saluja, "Yield-driven, false-path-aware clock skew scheduling," *IEEE Design & Test of Computers*, vol. 22, no. 3, pp. 214–222, 2005.

[37] C. Visweswariah, K. Ravindran, K. Kalafala, S. G. Walker, and S. Narayan, "First-order incremental block-based statistical timing analysis," in *Proceedings of the 41st annual design automation conference (DAC)*, 2004, pp. 331–336.

[38] G. Venkataraman, C. N. Sze, and J. Hu, "Skew scheduling and clock routing for improved tolerance to process variations," in *Proceedings of the asia and south pacific design automation conference (ASP-DAC)*, 2005, pp. 594–599.

[39] X. Wei, Y. Cai, and X. Hong, "Clock skew scheduling under process variations," in *Proceedings of the 7th international symposium on quality electronic design (ISQED)*, 2006, pp. 237–242.

[40] A. Agarwal, D. Blaauw, and V. Zolotov, "Statistical timing analysis for intra-die process variations with spatial correlations," in *Proceedings of the IEEE/ACM international conference on computer-aided design (ICCAD)*, 2003, pp. 900–907.

[41] H. Chang and S. S. Sapatnekar, "Statistical timing analysis under spatial correlations," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 24, no. 9, pp. 1467–1482, 2005.

[42] D. Blaauw, K. Chopra, A. Srivastava, and L. Scheffer, "Statistical timing analysis: From basic principles to state of the art," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 27, no. 4, pp. 589–607, 2008.

[43] M. Orshansky, L. Milor, P. Chen, K. Keutzer, and C. Hu, "Impact of spatial intrachip gate length variability on the performance of high-speed digital circuits," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 21, no. 5, pp. 544–553, 2002.

[44] Y. Liu, S. R. Nassif, L. T. Pileggi, and A. J. Strojwas, "Impact of interconnect variations on the clock skew of a gigahertz microprocessor," in *Proceedings of the 37th annual design automation conference (DAC)*, 2000, pp. 168–171.

[45] R. M. Karp, "A characterization of the minimum cycle mean in a digraph," *Discrete Mathematics*, vol. 23, no. 3, pp. 309–311, 1978.

[46] A. Dasdan and R. K. Gupta, "Faster maximum and minimum mean cycle algorithms for system-performance analysis," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 17, no. 10, pp. 889–899, 1998.

[47] W.-S. Luk, "Algorithms for design-for-manufacturability and network optimization: Lecture notes." Online course notes, https://luk036.github.io, 2026.

[48] K. A. Bowman, S. G. Duvall, and J. D. Meindl, "Impact of die-to-die and within-die parameter fluctuations on the maximum clock frequency distribution for gigascale integration," *IEEE Journal of Solid-State Circuits*, vol. 37, no. 2, pp. 183–190, 2002.

[49] A. F. Jenkinson, "The frequency distribution of the annual maximum (or minimum) values of meteorological elements," *Quarterly Journal of the Royal Meteorological Society*, vol. 81, no. 348, pp. 158–171, 1955.

[50] S. Coles, *An introduction to statistical modeling of extreme values*. London: Springer, 2001.

[51] T. D. Hansen and U. Zwick, "Lower bounds for Howard's algorithm for finding minimum mean-cost cycles," in *Proceedings of the 21st international symposium on algorithms and computation (ISAAC)*, in Lecture notes in computer science, vol. 6506. 2010.

[52] D. Gusfield, "Parametric combinatorial computing and a problem of program module distribution," *Journal of the ACM*, vol. 30, no. 3, pp. 551–563, 1983.

[53] P. J. Carstensen, "The complexity of some problems in parametric linear and combinatorial programming," PhD thesis, University of Michigan, 1983.

[54] K. Bringmann, T. D. Hansen, and S. Krinninger, "Improved algorithms for the minimum cost-to-time ratio cycle problem," in *Proceedings of the 44th international colloquium on automata, languages, and programming (ICALP)*, in LIPIcs, vol. 80. 2017.

[55] W.-S. Luk, "digraphx: Directed-graph algorithms for network optimization." Reference implementations in Python, C++, and Rust (siblings `digraphx`, `digraphx-cpp`, `digraphx-rs`), https://github.com/luk036/digraphx, 2026.

[56] W.-S. Luk, "netoptim: Network-flow optimization oracles and cutting-plane solvers." Reference implementations in Python, C++, and Rust (siblings `netoptim`, `netoptim-cpp`, `netoptim-rs`), https://github.com/luk036/netoptim, 2026.

[57] K. Ravindran, A. Kuehlmann, and E. Sentovich, "Multi-domain clock skew scheduling," in *Proceedings of the IEEE/ACM international conference on computer-aided design (ICCAD)*, 2003, pp. 801–808.